



Machine Learning II

Data Science M.Sc.

Prof. Dr. Steffen Wagner
Berliner Hochschule für Technik

Winter term 2025/26



Introduction to Artificial Neural Networks



Introduction to Artificial Neural Networks< (NN)

- ▶ The terms *Artificial Neural Network* and *Deep Learning* are used by many interchangeably. There are small differences in the meanings, but we will not make a distinction in this course.
- ▶ The course notes will use the term Artificial Neural Network with the abbreviation **NN** throughout.
- ▶ James et al. has a Chapter called *Deep Learning* (Ch. 10), which is new to the second edition. This ML2 course doesn't follow exactly the content in James, but there is significant overlap and the two are presented at the roughly same level of theory.



Agenda NN

- ▶ Neural Networks and Deep Learning are (probably) not new to you. Other parts of the Data Science programme concentrate more on practical aspects, constructing large NN configurations to maximise the prediction capabilities.
- ▶ This course will instead concentrate on the basics and the theory behind neural networks. The subjects covered in the following lectures of the course are:
 - ▶ Introduction
 - ▶ Optimisation Methods
 - ▶ Mathematical and practical aspects of NNs



Data Structure and notation

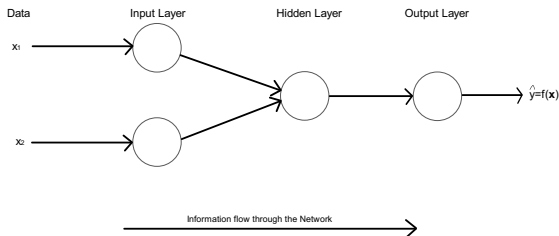
- ▶ Artificial Neural Networks (NNs) can be used for both regression problems and classification problems. We'll start with *regression* because it is easier.
- ▶ A data Matrix $\mathbf{X}$ has n rows (the elements or observations) and p columns (variables).
- ▶ We conceptualise the NN as processing each observation (row) one after the other. For an arbitrary observation we will drop the row index and represent it as a p -dimensional vector $\mathbf{x}$.
- ▶ The predicted value for the same arbitrary observation $\mathbf{x}$ is $f(\mathbf{x})$.
- ▶ The outcome variable for $\mathbf{X}$ is $\mathbf{y}$, a vector of n observations.
The observed outcome value for the arbitrary observation $\mathbf{x}$ is y .
- ▶ The *cost* or *loss* for one observation $\mathbf{x}$ is $L = (y - f(\mathbf{x}))^2$
- ▶ When we consider all of the observations, the index $i = 1, \dots, n$ will be used.



First Neural Network

Each NN can be visualised using a network graph. A very simple (feed-forward) NN for introductory purposes is:

Diagram 1



- ▶ This NN has **3 Layers**: an *input layer* which receives the data for observation x , a *hidden layer* and an *output layer* whose output value is the predicted value $f(x)$, which hopefully is a value close to y .
- ▶ There are two nodes in the input layer ($p=2$, i.e. two variables in the input data), just one node in the hidden layer and one node in the output layer.



Information flow

1. The hidden layer node receives a weighted sum of the data and a constant is added (called a bias).

$$z_1^{(1)} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}.$$

2. This value is then transformed by a *non-linear* function $\sigma(\cdot)$.
3. The hidden layer node outputs a value, which is called the *activation* of the node.

$$a_1^{(1)} = \sigma(z_1^{(1)}) = \sigma(w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}).$$

4. The output layer in this example receives activation from the hidden layer multiplies it by a weight $w_{11}^{(2)}$ and adds another bias. The result is then transformed using another function $g_1(\cdot)$:

$$a_1^{(2)} = g_1(w_{11}^{(2)} a_1^{(1)} + b_1^{(2)}).$$

5. The fitted/predicted value is the activation at the output layer.

$$f(\mathbf{x}) = a_1^{(2)}$$



Remarks regards notation

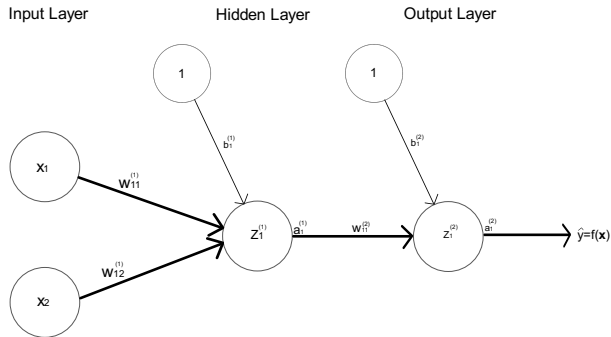
- ▶ The superscript $(\cdot)$ indicates which layer we are dealing with. The zero-th layer is the input layer. The last layer L is the output layer and all layers in between are hidden layers.
- ▶ The ordering of the subscripts for the weights seem misleading.
 $w_{12}^{(k)}$ joins the *second* node in layer $k-1$ with the *first* node in layer k . The reason for this is it keeps the notation for matrix multiplication simpler.
- ▶ Don't get confused by L being used for the amount of layers and the loss function:
 - ▶ Whenever L is a function (e.g. being differentiated) we are considering loss.
 - ▶ Whenever L appears as a number (e.g. index/superscript for weights) it corresponds to a layer.



Diagramm with coefficients

The NN can now be graphically represented with coefficients.

Diagram 2

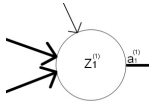




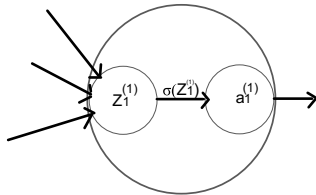
The nodes in detail

Each hidden and output layer node is represented as one circle, but carries out two transformations, which could be represented by two circles.

Diagram 3

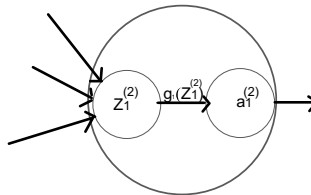


Hidden Layer



$\sigma(\cdot)$ is the activation function

Output Layer





Activation functions

A common activation function $\sigma(\cdot)$ is the sigmoid or inverse logistic function

$$\sigma(v) = \frac{1}{1 + e^{-v}}.$$

In early neural networks, the activation function was the so called *heaviside* function

$$\sigma(v) = \begin{cases} 1 & \text{for } v > 0 \\ 0 & \text{for } v \leq 0. \end{cases}$$

The node is “activated” if the combination of the inputs exceed the threshold zero. This is a simple model for how neurons in the brain function, hence the name artificial neural network.

For regression the identity function is often used as the output function

$$g_1(v) = v.$$



A simple numerical example

Let $x_1 = 4$, $x_2 = 7$ and $y = 4.1$.

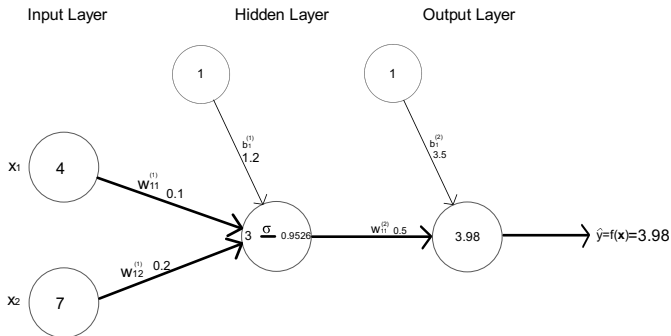
- ▶ The weights for the first node are $w_{11}^{(1)} = 0.1$, $w_{12}^{(1)} = 0.2$ and $b_1^{(1)} = 1.2$
- ▶ The weights for the second node (output layer) are $w_{11}^{(2)} = 0.5$ and $b_1^{(2)} = 3.5$
- ▶ The activation function is the sigmoid function, and their output function is the identity function.
- ▶ The weighted inputs are $z_1^{(1)} = 0.1 \times 4 + 0.2 \times 7 + 1.2 = 3$.
- ▶ The activation of the hidden layer node is $a_1^{(1)} = \sigma(3) = \frac{1}{1 + e^{-3}} = 0.9526$

The output is then $a_1^{(2)} = g_1(0.5 \times 0.9526 + 3.5) = 3.976$



Visualization

Diagram 4





Neural Nets and model parameters

It is important to note that all the weights and biases are the parameters of the NN model.

These values are applied to *every observation* in the (training) data set.

If the second observation were $x_1 = 3$, $x_2 = -1$ and $y = 4.1$, we would obtain:

- ▶ $z_1^{(1)} = 1.3$
- ▶ $a_1^{(1)} = 0.7858$
- ▶ $a_1^{(2)} = 3.892$, $\Rightarrow 3.892$ is the second fitted value.



Parameter estimation (= Fitting)

- ▶ As with all machine learning methods we need to fit or train the model to the data.
- ▶ This is done in the usual way by minimising a loss function, which for NN-regression is the sum of squared errors SSE over the training set.

$$SSE = \sum_{i \in Tr} L_i = \sum_{i \in Tr} (y_i - f(\mathbf{x}_i))^2$$

Where Tr is the training data set.

- ▶ The minimisation is done over all parameters $w_{11}^{(1)}$, $w_{12}^{(1)}$, $b_1^{(1)}$, $w_{11}^{(2)}$ and $b_1^{(2)}$.
- ▶ Note that the number of nodes in each layer and the activation/output function are hyperparameters chosen by the user and are not part of the minimisation.



Training validation and test data

- ▶ At the moment we are only considering the most simple neural networks. In practice the the NN model can be extended in many ways e.g more nodes in the first hidden layer, more hidden layers, different choice of activation and output functions.
- ▶ The complexity of these models means that it is very easy to fit the data very well with the result that the prediction for future data is poor.
- ▶ Therefore, it is particularly important that the data are split into training and validation data sets. Very often we want a final assessment for the accuracy of model, in which case a test data set is needed as well.



Training validation and test data (cont)

- ▶ The optimization of the parameters $w_{11}^{(1)}$, $w_{12}^{(1)}$, $b_1^{(1)}$, $w_{11}^{(2)}$ and $b_1^{(2)}$ for a fixed model design is done using the training data.
- ▶ If you want to make a choice about hyperparameters, such as how many nodes to fit in the first hidden layer, this is done by fitting each model on the training data, and choosing the hyperparameters that minimise the loss of the **validation data**.
- ▶ Reporting benchmark statistics (such as the accuracy rate of a classifier) or comparing different machine learning methods must be done using “clean data” i.e. the **test data**.
- ▶ Note that if the data are only split into two data sets, then data on which the model is validated are often called the test data!



Scaling

- ▶ Although not totally necessary, fitting a neural network is much faster if the data are scaled (also called normalised) first.
- ▶ Each variable is scaled by subtracting that variable's *training data* mean and dividing by that variable's *training data* standard deviation.

There are two common methods of scaling.

- ▶ The first is standardisation, i.e. *mean and sd* scaling: for variable x_j

$$x'_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j} \quad \text{with } \bar{x}_j = \frac{1}{|Tr|} \sum_{i \in Tr} x_{ij} \quad \text{and } s_j = \frac{1}{|Tr| - 1} \sum_{i \in Tr} (x_{ij} - \bar{x}_j)^2$$

- ▶ The second is normalisation to the intervall $[0, 1]$:

$$x'_{ij} = \frac{x_{ij} - \min\{x_j\}}{\max\{x_j\} - \min\{x_j\}}$$



Scaling (cont)

- ▶ The output variable (for regression) is also scaled using the same method.
- ▶ When scaling or unscaling the validation/test data, you use the same min/max/mean/sd **as used on the training data**. You will get worse results if you do not do this.
- ▶ We will usually assume the data x_{ij} has already been scaled and omit the '.



Parameter estimation and algorithms

- ▶ In the introductory very simple example we already have 5 parameters to vary in order to minimise the loss, here SSE. This is a five dimensional **function minimisation** problem.
- ▶ We can use any function minimisation algorithm to search for the best values for the 5 parameters.
- ▶ For introductory purposes, we will use a very simple minimisation routine so that you we better understand the mathematical principles of the NN Model.
Remark: this minimisation routine converges very slowly.
- ▶ Later we will discuss more performant minimisation algorithms.
- ▶ The “basic” algorithm for NNs is the steepest descent algorithm using *back propagation*. Recently other minimisation algorithms have been developed but most are based around speeding up the steepest descent/back propagation method.

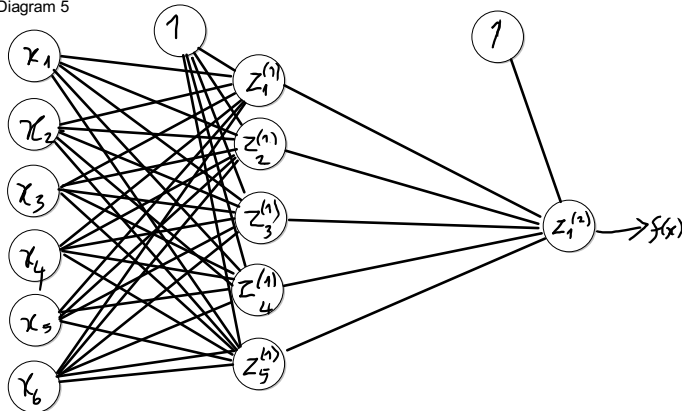


Model complexity

There is never just one node in the hidden layer. Even with small data sets it is common to use $M = 5$ to 20 nodes.

Example and notation for one hidden layer with $M = 5$ units and $p = 6$.

Diagram 5





Model complexity

Parameters to be estimated:

- ▶ Each edge has a weight, and there is a bias for each Node in the first and second layer.
- ▶ The general number of parameters (for a model with one hidden Layer) is

$$(p + 1)M + (M + 1) = (p + 2)M + 1.$$

- ▶ Example: the network on the previous slide has 41 edges. Each edge has a weight which needs to be optimised!

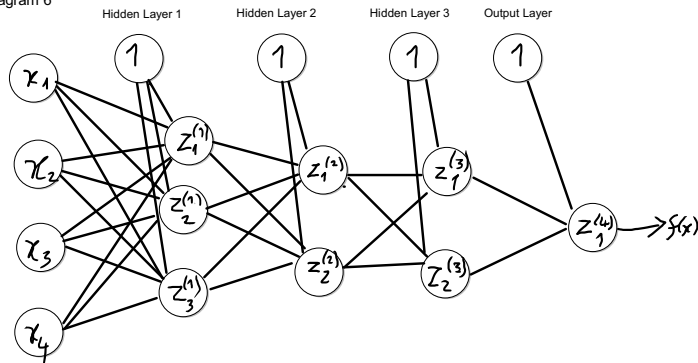
Fitting a neural network requires many settings to be tweaked. The aim is to find a combination of parameters which gives a good output.



Multiple hidden layers

Instead of adding more nodes to one hidden layer, we can add more hidden layers.
 Example: 3 hidden layers $L=4$, $M_1=3$, $M_2=2$, $M_3=2$ and $p=4$.

Diagram 6





The output layer

So far we have used one output node with:

$$a_1^{(L)} = g_1 \left(\sum_{j=1}^{M_{L-1}} w_{1j}^{(L)} a_j^{(L-1)} + b_1^{(L)} \right).$$

The outcome variable can be multidimensional in which case there are as many output nodes as there are dimensions in y .

Example: if the outcome variable consists of the two standard blood pressure measurements, systolic and diastolic, then a NN predicting these outcomes requires two output nodes

$$a_1^{(L)} = g_1 \left(\sum_{j=1}^{M_{L-1}} w_{1j}^{(L)} a_j^{(L-1)} + b_1^{(L)} \right) \text{ and } a_2^{(L)} = g_2 \left(\sum_{j=1}^{M_{L-1}} w_{2j}^{(L)} a_j^{(L-1)} + b_2^{(L)} \right).$$

The predictor function is an $\mathbb{R}^p \rightarrow \mathbb{R}^2$ function $f(\mathbf{x}) = (a_1^{(L)}, a_2^{(L)})$.



Neural network classifiers

Classification NNs have the same configuration for the input and hidden layers, but the output layer takes account of the fact that the output variable takes one of K values.

Examples:

- ▶ $K=2$ Binary classifier, usually internally coded as $\{0, 1\}$.
- ▶ $K=3$ classifier: Iris dataset.
- ▶ $K=10$ classifier: digits 0 to 9.
- ▶ $K=62$ classifier: upper case letters, lower case letters and digits 0 to 9.

In K -class classification there are K output nodes one for each class.

Important: To handle K -class scenarios we redefine y as a K -dimensional binary vector.



Neural network classifiers

- ▶ In NN-regression we used the identity function at the output layer.
- ▶ For NN-classifiers the usual output function is the sigmoid function, so

$$g_k(v) = \sigma(v)$$

- ▶ The activation of each output node is

$$a_k^{(L)} = \sigma \left(\sum_{j=1}^{M_{L-1}} w_{kj}^{(L)} a_j^{(L-1)} + b_1^{(L)} \right)$$

- ▶ The resulting activation is a number between 0 and 1.



From activations to predictions

The K activations are numbers between 0 and 1. But how do we obtain our prediction?

The answer depends on what we actually want.

The *predicted value* is the class number k with the largest activation value.

$$f(\mathbf{X}) = \operatorname{argmax}_k \left\{ a_k^{(L)} \right\}$$

If instead we want to predict the probabilities for each class $\pi_k(\mathbf{x})$, we need to standardise the activations, because they have to add up to one.

$$\pi_k(\mathbf{X}) = \frac{a_k^{(L)}}{\sum_{k=1}^K a_k^{(L)}}.$$



Loss function for classification

- ▶ To fit the NN classifier we minimise a loss function, comparing the K output values $a_k^{(L)}$ with the one observed value y .
- ▶ As with NN-regression, we can use the squared error loss for the loss in the i -th observation:

$$L_i = \sum_{k=1}^K (\pi_k(\mathbf{x}_i) - y_{ik})^2$$

- ▶ Alternatively and more common, the cross-entropy¹ aka deviance is used as the loss function more often than the squared error loss:

$$L_i = - \sum_{k=1}^K y_{ik} \log(\pi_k(\mathbf{x}_i))$$

- ▶ By using the continuous values of $\pi_k(\cdot)$ instead the binary predictions, the loss function is continuous, so it's much easier to minimise using numerical methods.

¹also used as criteria in the context of classification trees



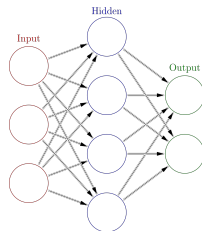
Optimisation in artificial neural networks



Recap

An artificial neural network (NN) consist of the following layers:

- ▶ The input layer which consists of one node for each predictor variable.
- ▶ Hidden layers, which combine the values of the nodes at the previous layer.
- ▶ An output layer combines the values of the nodes at the last hidden layer.



For a regression NN with 1 output variable, the output layer is one node.

For a classifier NN there is one node in the output layer for each class.

source: [https://en.wikipedia.org/wiki/Neural_network_\(machine_learning\)](https://en.wikipedia.org/wiki/Neural_network_(machine_learning))



What's new?

- ▶ The NN is defined by the structure of the network and(!) the values of the weights and biases, which in comparison to other ML methods is a large number of parameters.
- ▶ The parameters should be chosen to minimise an appropriate loss function.
- ▶ The method of minimisation used in practise is **back propagation**.²

²In recent years other minimisation methods have been developed for neural networks but these build on the principles of back propagation.



Optimisation

Minimisation: find the point x such that $f(x)$ reaches its smallest value.

Maximisation: find the point x such that $f(x)$ reaches its largest value.

Remarks:

- ▶ As maximising $f(x)$ is exactly the same as minimising $-f(x)$, we usually consider just the minimisation case and call the procedure **optimisation**.
- ▶ x can be a scalar variable (1-dim) or a vector in $\mathbb{R}^n$ (n -dim)
- ▶ Most numerical optimisation methods find a local optimum rather than a global optimum.



One dimensional optimisation

Simple maths: Let $y = f(x) = 3x^4 + 5x^3 - 20x^2 + 8x + 10$

The classic approach to find the extreme points of f is to obtain the derivative

$$\frac{dy}{dx} = f'(x) = 12x^3 + 15x^2 - 40x + 8$$

set this equal to zero to find the roots, since an extreme point x_0 of a continuous function has the property $f'(x_0) = 0$.

Q: What if we can't solve the first order condition explicitly?



One dimensional optimisation

A numeric algorithm which uses $f'(x_0) = 0$:

1. Choose a starting point x_0 and a step size s . Set $i = 0$.
2. Calculate $\frac{dy}{dx} \Big|_{x=x_i} = f'(x_i)$. This is the 1-dim *gradient* of $f(x)$ at x_i .
3. Move in downhill direction an amount $\Delta = s \cdot f'(x_0)$. I.e. $x_{i+1} = x_i - \Delta = x_i - s \cdot f'(x_i)$.
4. Iterate until x_i converges.



One dimensional optimisation

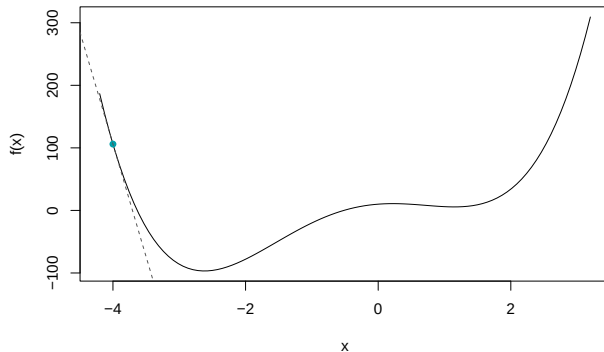
Remarks:

- ▶ A positive $f'(x)$ “points uphill”. Since we wish to minimise $f(x)$ we consider $-f'(x)$.
- ▶ This method finds a local minimum. Finding a global minimum is a much harder problem.
- ▶ If $f'(x)$ is small, the update will not move very far. This causes a problem when near to the optimal solution and when near an inflection point/saddle point.
- ▶ We either need to know $f'(x)$ or be able to numerically estimate it. The former is better if this is possible.
- ▶ The step size s does not have to be static, it can be *adaptive*, i.e. change with iteration number, depending on the properties of f .



One dimensional optimisation

Demo: Starting point $x_0 = -4$

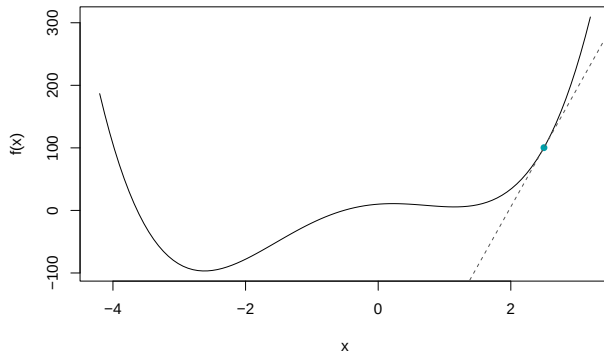


Use Acrobat Reader to run the animations, if you have problems in your default pdf viewer.



One dimensional optimisation

Demo: Starting point $x_0 = 2.5$



Use Acrobat Reader to run the animations, if you have problems in your default pdf viewer.



Newton-Raphson method

- ▶ A well known version of this optimisation method is the **Newton-Raphson Method** which sets the step size to $s_i = \frac{1}{f''(x_i)}$.
- ▶ This of course means that the second derivative must also be known.
- ▶ Numerically estimating the second derivative is possible but comes with a significant computational cost. The estimation can sometimes be unstable.



Two dimensional optimisation

We consider a function that takes two inputs and gives one output $f(x_1, x_2) = f(\mathbf{x})$.

Goal: find the minimum of $f(\mathbf{x})$.

- ▶ A local minimum has the property that both *partial derivatives* are zero:

$$\frac{\partial f}{\partial x_1} = 0 \quad \text{and} \quad \frac{\partial f}{\partial x_2} = 0.$$

- ▶ The vector of partial derivatives of f is $\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{pmatrix}$ and is called “gradient f ”.
- ▶ The minimisation algorithm is similar to the one dimensional case, but now downhill is specified by the vector $-\nabla f(\mathbf{x})$



Steepest descent algorithm

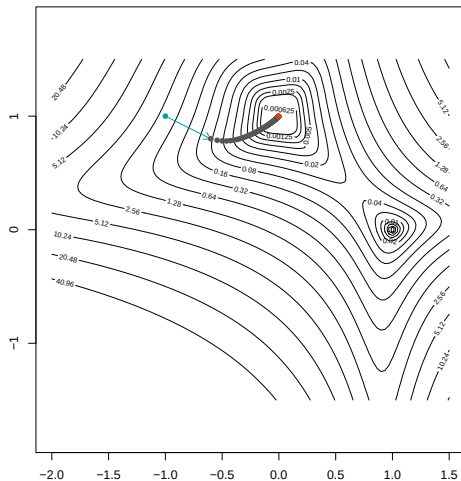
Let $f(\mathbf{x})$ be the 2-dim function to be minimised.

1. Determine the partial derivatives of f .
2. Choose a starting point $\mathbf{x}_0$, and a step size s . Set $i = 0$.
3. Calculate $\nabla f(\mathbf{x}_i)$.
4. Move downhill an amount $s\nabla f(\mathbf{x}_i)$. I.e. $\mathbf{x}_{i+1} = \mathbf{x}_i - s\nabla f(\mathbf{x}_i)$.
5. Iterate until $\mathbf{x}_i$ converges.



Two dimensional optimisation: example

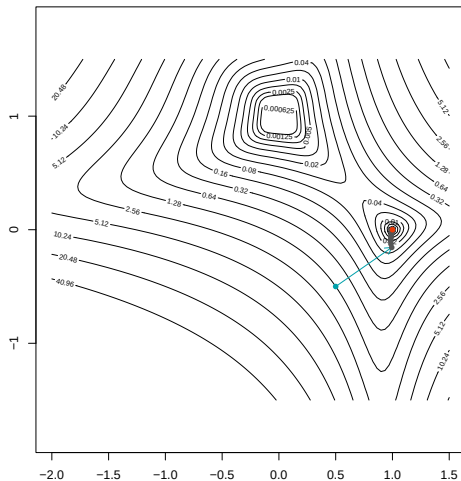
Starting point: $\mathbf{x} = (-1, 1)^\top$





Two dimensional optimisation: example

Starting point: $\mathbf{x} = (0.5, -0.5)^T$





Higher dimensional steepest descent

- ▶ Many machine learning algorithms involve minimising a function associated with a model that has many parameters $f(\theta)$, with $\theta = (\theta_1, \theta_2, \dots, \theta_{n_\theta})$.
- ▶ In some special cases there is a non-iterative procedure which gives the optimal parameters directly. In most other cases the minimisation has to be done using an iterative numerical procedure.
- ▶ It would be good if we can obtain the partial derivatives for each element of θ , i.e. $\nabla f(\theta)$.
- ▶ Unfortunately this is often not the case and the matrix of second derivatives has to be estimated.
- ▶ In the common situation that f is a sum of squared error loss function the Newton-Raphson optimisation is a good algorithm to use.
- ▶ For neural networks we are in luck! With a bit of work we can specify $\nabla f(\theta)$, using a method called **back propagation**. The work invested is rewarded by a more efficient algorithm.



Back propagation: the principle

General idea:

- ▶ We will assume a regression NN with a squared error loss function.³
- ▶ The NN loss function $L(\theta)$ depends on all our weights and biases represented by parameter as a vector θ .

$$L(\theta) = \sum_{i \in Tr} (y_i - f(\mathbf{x}_i; \theta))^2$$

- ▶ We are allowed to choose the elements of θ .
- ▶ $\mathbf{x}_i$ and y_i are fixed: this is our data!
- ▶ There maybe a tons of parameters: n_θ can easily be > 1000 .
- ▶ Just as in the 2 dimensional function minimisation we will look for the direction with the steepest descent, that is the vector

$$-\nabla L(\theta)$$

as this decreases the loss function most efficiently.

³Adaptations to classification and other loss functions use the same principles.



Back propagation: the principle

- ▶ Each element of $-\nabla L(\theta)$ tells us how sensitive the loss function is to the corresponding parameter:

Lets assume a NN with 9 Parameters, then

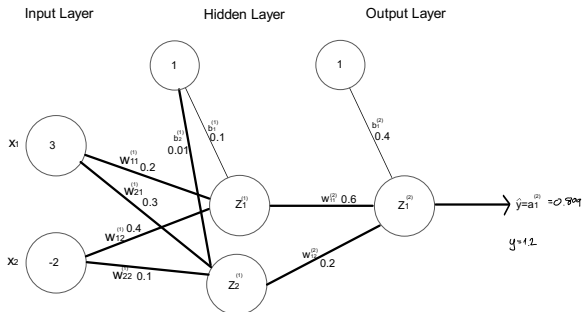
$$-\nabla L(\theta) = -\nabla L(\theta_1, \theta_2, \dots, \theta_9) = - \begin{pmatrix} \frac{\partial L}{\partial \theta_1} \\ \frac{\partial L}{\partial \theta_2} \\ \vdots \\ \frac{\partial L}{\partial \theta_9} \end{pmatrix}$$

- ▶ $-\nabla L(\theta)$ is vector with as many dimensions as there are parameters in the NN.
- ▶ This vectors points into the direction in which we have to change the NN parameters to reach *as fast as possible* the (local) minimum of model mismatch L .



Back propagation: example

Diagram 1



The data for the first element is $x_{11} = 3$ and $x_{12} = -2$ with outcome value $y_1 = 1.2$



Back propagation: example

The NN is given as

$$\begin{aligned} f(\theta^{(1)}, \theta^{(2)}, \mathbf{x}_i) &= g_1 \left(w_{11}^{(2)} a_1^{(1)} + w_{12}^{(2)} a_2^{(1)} + b_1^{(2)} \right) \quad | \text{ for regression: } g_1(v) = v \\ &= w_{11}^{(2)} a_1^{(1)} + w_{12}^{(2)} a_2^{(1)} + b_1^{(2)} \end{aligned}$$

$$\begin{aligned} \text{]with } a_1^{(1)}(\theta_1^{(1)}) &= \sigma \left(w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)} \right) \quad | \text{ with: } \sigma(v) = \frac{1}{1 + \exp(-v)} \\ a_2^{(1)}(\theta_2^{(1)}) &= \sigma \left(w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)} \right) \end{aligned}$$

and the loss function as $L(\theta) = L(\theta_1, \dots, \theta_9) = \sum_{i \in Tr} (y_i - f(\theta, \mathbf{x}_i))^2$.

we see:

- ▶ Quadratic relationship between $f(\theta^{(1)}, \theta^{(2)}, \mathbf{x}_i)$ and $L(\theta)$
- ▶ **output layer** parameters $\theta^{(2)}$ **directly** impact f
- ▶ **hidden layer** parameters $\theta^{(1)}$ **indirectly** impact f .



Recap: chain rule for differentiation

Before we look at the example, we should do a little revision of the chain rule in calculus.

- ▶ There are two notation systems for differentiation, both are useful in different ways.
- ▶ One notation is to express a function as $f(x)$ and the derivative as $f'(x)$ (Lagrange notation)
- ▶ The “Leibniz notation” expresses the value of the function also as $f(x)$ (e.g. $f(x) = 2x^2 + 4$) and the derivative of $f(x)$ with respect to x is $\frac{df(x)}{dx}$
- ▶ For the chain rule the Leibniz notation is much easier to follow, especially when the chain has many links.



Chain rule with two links

Let f be a function of x .

If we can express f as a function of v , and v as a function of x , then the derivative of f w.r.t.⁴ x is

$$\frac{df}{dx} = \frac{df}{dv} \frac{dv}{dx}$$

Usually the two derivatives on the right will be easier to determine than the one on the left directly.

In the Lagrange notation we write $f(x) = u(v(x))$, and the chain rule is

$$f'(x) = u'(v(x)) v'(x).$$

⁴w.r.t. means “with respect to”



Example of the chain rule

Let $L(f) = (y - f)^2$ and we wish to find $\frac{dL}{df}$.

Put $u = (y - f)$ and we obtain

$$L = u^2 \quad \frac{dL}{du} = 2u \quad \frac{du}{df} = -1$$

and therefore

$$\frac{dL}{df} = \frac{dL}{du} \frac{du}{df} = -2u$$

$$\frac{dL}{df} = -2(y - f)$$



Chain rule with three links

Express f as a function of u and u as a function of v and v as a function of x .

The derivative of f w.r.t. x is

$$\frac{df}{dx} = \frac{df}{du} \frac{du}{dv} \frac{dv}{dx}$$

In the Lagrange notation we write $f(x) = w(u(v(x)))$, and the chain rule is

$$\Rightarrow f'(x) = w'(u(v(x)))u'(v(x))v'(x).$$



Back propagation: example

Now let's apply the chain rule to calculate $-\nabla L(\theta)$ with

$$L(\theta) = L(\theta_1, \dots, \theta_9) = \sum_{i \in Tr} (y_i - f(\theta, \mathbf{x}_i))^2.$$

We consider the loss for one arbitrary observation neglecting the index i

$$\begin{aligned} L &= (y - f(\mathbf{x}, \theta))^2 \\ &= \left(y - \left(w_{11}^{(2)} a_1^{(1)} + w_{12}^{(2)} a_2^{(1)} + b_1^{(2)} \right) \right)^2 \\ &= \left(y - \left(w_{11}^{(2)} \sigma \left(w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)} \right) + w_{12}^{(2)} \sigma \left(w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)} \right) + b_1^{(2)} \right) \right)^2 \end{aligned}$$

Applying the chain rule, we can differentiate $L(\theta^{(1)}, \theta^{(2)})$ w.r.t. to the 9 NN parameters.



Back propagation: example

Let's start simple having a look at the parameters $\theta^{(2)}$ of the output node:

$$(1) \quad \frac{\partial L}{\partial w_{11}^{(2)}} = \frac{\partial L}{\partial f} \frac{\partial f}{\partial w_{11}^{(2)}} = \underbrace{-2 \cdot (y - f(\mathbf{x}, \theta))}_{\frac{\partial L}{\partial f}} \cdot \underbrace{a_1^{(1)}}_{\frac{\partial f}{\partial w_{11}^{(2)}}}$$

$$(2) \quad \frac{\partial L}{\partial w_{12}^{(2)}} = \frac{\partial L}{\partial f} \frac{\partial f}{\partial w_{12}^{(2)}} = \underbrace{-2 \cdot (y - f(\mathbf{x}, \theta))}_{\frac{\partial L}{\partial f}} \cdot \underbrace{a_2^{(1)}}_{\frac{\partial f}{\partial w_{12}^{(2)}}}$$

$$(3) \quad \frac{\partial L}{\partial b_1^{(2)}} = \frac{\partial L}{\partial f} \frac{\partial f}{\partial b_1^{(2)}} = \underbrace{-2 \cdot (y - f(\mathbf{x}, \theta))}_{\frac{\partial L}{\partial f}} \cdot \underbrace{1}_{\frac{\partial f}{\partial b_1^{(2)}}}$$

That's it. We calculated the elements of $\nabla L(\theta^{(2)})$ and can later⁵ calculate the numeric values for the data y_i and $\mathbf{x}_i$ in our training set.

⁵slides 57 ff



Back propagation: example

Differentiation w.r.t. the parameters of the hidden layer $\theta^{(1)}$ requires more steps, since they impact the observed loss according the information flow in the network more indirectly:

$$\theta^{(1)} \longrightarrow z^{(1)} \longrightarrow a^{(1)} \longrightarrow z^{(2)} \longrightarrow L$$

which can be expressed as

$$L_i = \left(y_i - f \left(x_i, a^{(1)} \left(z^{(1)} \left(\theta^{(1)} \right) \right), \theta^{(2)} \right) \right)^2.$$

Therefore we have to consider four derivatives each representing one arrow in the upper diagram

$$\frac{\partial L}{\partial \theta^{(1)}} = \frac{\partial L}{\partial f} \frac{\partial f}{\partial a^{(1)}} \frac{\partial a^{(1)}}{\partial z^{(1)}} \frac{\partial z^{(1)}}{\partial \theta^{(1)}}$$

Back propagation uses the derivatives for each transition to quantify the impact of initial parameters onto L back through the network.



Back propagation: example

Let's consider the parameters $\theta_1^{(1)} = \left(w_{11}^{(1)}, w_{12}^{(1)}, b_1^{(1)} \right)^T$ of node 1 in the hidden layer:

$$(1) \quad \frac{\partial L}{\partial f} = -2(y - f(\mathbf{x}, \theta))$$



Back propagation: example

Let's consider the parameters $\theta_1^{(1)} = \left(w_{11}^{(1)}, w_{12}^{(1)}, b_1^{(1)} \right)^T$ of **node 1** in the hidden layer:

$$(1) \quad \frac{\partial L}{\partial f} = -2(y - f(\mathbf{x}, \theta))$$

$$(2) \quad \frac{\partial f}{\partial a_1^{(1)}} = \frac{\partial}{\partial a_1^{(1)}} \left(w_{11}^{(2)} a_1^{(1)} + w_{12}^{(2)} a_2^{(1)} + b_1^{(2)} \right) = w_{11}^{(2)}$$



Back propagation: example

Let's consider the parameters $\theta_1^{(1)} = \left(w_{11}^{(1)}, w_{12}^{(1)}, b_1^{(1)} \right)^T$ of **node 1** in the hidden layer:

$$(1) \quad \frac{\partial L}{\partial f} = -2(y - f(\mathbf{x}, \theta))$$

$$(2) \quad \frac{\partial f}{\partial a_1^{(1)}} = \frac{\partial}{\partial a_1^{(1)}} \left(w_{11}^{(2)} a_1^{(1)} + w_{12}^{(2)} a_2^{(1)} + b_1^{(2)} \right) = w_{11}^{(2)}$$

$$(3) \quad \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} = \frac{\partial}{\partial z_1^{(1)}} \left(\sigma \left(z_1^{(1)} \right) \right) = \frac{\partial}{\partial z_1^{(1)}} \left(\frac{1}{1 + \exp \left(-z_1^{(1)} \right)} \right) = \frac{\exp \left(-z_1^{(1)} \right)}{\left(1 + \exp \left(-z_1^{(1)} \right) \right)^2}$$



Back propagation: example

Let's consider the parameters $\theta_1^{(1)} = \left(w_{11}^{(1)}, w_{12}^{(1)}, b_1^{(1)} \right)^T$ of **node 1** in the hidden layer:

$$(1) \quad \frac{\partial L}{\partial f} = -2(y - f(\mathbf{x}, \theta))$$

$$(2) \quad \frac{\partial f}{\partial a_1^{(1)}} = \frac{\partial}{\partial a_1^{(1)}} \left(w_{11}^{(2)} a_1^{(1)} + w_{12}^{(2)} a_2^{(1)} + b_1^{(2)} \right) = w_{11}^{(2)}$$

$$(3) \quad \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} = \frac{\partial}{\partial z_1^{(1)}} \left(\sigma \left(z_1^{(1)} \right) \right) = \frac{\partial}{\partial z_1^{(1)}} \left(\frac{1}{1 + \exp \left(-z_1^{(1)} \right)} \right) = \frac{\exp \left(-z_1^{(1)} \right)}{\left(1 + \exp \left(-z_1^{(1)} \right) \right)^2}$$

$$(4) \quad \frac{\partial z_1^{(1)}}{\partial \theta_1^{(1)}} = \begin{pmatrix} \frac{\partial}{\partial w_{11}^{(1)}} \left(w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)} \right) \\ \frac{\partial}{\partial w_{12}^{(1)}} \left(w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)} \right) \\ \frac{\partial}{\partial b_1^{(1)}} \left(w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)} \right) \end{pmatrix} = \begin{pmatrix} x_1 \\ x_2 \\ 1 \end{pmatrix}$$

That's it. Determining the four partial derivatives for the parameters $\theta_2^{(1)}$ of **node 2** in the hidden layer works analogously. Only equations (2) and (3) change. How?



Back propagation: example

What remains is calculating the nine elements of $-\nabla L(\theta)$.

We require as input for the derived functional relationships

- ▶ the predicted value for this observation: $f(\mathbf{x}_i, \theta)$
- ▶ The activations of the two nodes in the hidden layer: $a_1^{(1)}$ and $a_2^{(1)}$
- ▶ The weighted sums in the two nodes of the hidden layer: $z_1^{(1)}$ and $z_2^{(1)}$

based on the training data $\mathbf{x}_i$ and y_1 and the current parameters θ .



Back propagation: example

The scenario given in slide 46 is as follows

- ▶ data: $\mathbf{x}_1 = (3, -2)$ and $y_1 = 1.2$
- ▶ parameter

$$\boldsymbol{\theta}^\top = (w_{11}^{(1)} = 0.2, w_{12}^{(1)} = 0.4, b_1^{(1)} = 0.1, w_{21}^{(1)} = 0.3, w_{22}^{(1)} = 0.1, b_2^{(1)} = 0.01, w_{11}^{(2)} = 0.6, w_{12}^{(2)} = 0.2, b_1^{(2)} = 0.4)$$

we obtain

- ▶ $f(\mathbf{x}_i, \boldsymbol{\theta}) = 0.819$
- ▶ $z_1^{(1)} = -0.1$ and $z_2^{(1)} = 0.71$
- ▶ $a_1^{(1)} = \sigma(z_1^{(1)}) = 0.475$ and $a_2^{(1)} = \sigma(z_2^{(1)}) = 0.67$

Based on these values we now calculate all 9 elements of $-\nabla L(\boldsymbol{\theta})$ using the results from slides 53 and 55.



Back propagation: example

Let's start with $\theta_1^{(1)}$ using equations (1) - (4) from slide 55:

$$(1) \quad \frac{\partial L}{\partial f} = -2(y - f(\mathbf{x}, \theta)) = -2(1.2 - 0.819) = -0.762$$

$$(2) \quad \frac{\partial f}{\partial a_1^{(1)}} = w_{11}^{(2)} = 0.6$$

$$(3) \quad \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} = \frac{\exp(-z_1^{(1)})}{\left(1 + \exp(-z_1^{(1)})\right)^2} = \frac{\exp(-(-0.1))}{(1 + \exp(-(-0.1)))^2} = 0.249$$

$$(4) \quad \frac{\partial z_1^{(1)}}{\partial \theta_1^{(1)}} = \begin{pmatrix} x_1 \\ x_2 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ -2 \\ 1 \end{pmatrix}$$

In summary we obtain:

$$\frac{\partial L}{\partial \theta_1^{(1)}} = -0.762 \cdot 0.6 \cdot 0.249 \cdot \begin{pmatrix} 3 \\ -2 \\ 1 \end{pmatrix} = \begin{pmatrix} -0.342 \\ 0.228 \\ -0.114 \end{pmatrix}$$



Back propagation: example

Next we consider $\theta_2^{(1)}$ in the same way:

$$(1) \quad \frac{\partial L}{\partial f} = -2(y - f(\mathbf{x}, \theta)) = -2(1.2 - 0.819) = -0.762$$

$$(2) \quad \frac{\partial f}{\partial a_2^{(1)}} = w_{12}^{(2)} = 0.2$$

$$(3) \quad \frac{\partial a_2^{(1)}}{\partial z_2^{(1)}} = \frac{\exp(-z_2^{(1)})}{\left(1 + \exp(-z_2^{(1)})\right)^2} = \frac{\exp(-0.71)}{(1 + \exp(-0.71))^2} = 0.221$$

$$(4) \quad \frac{\partial z_2^{(1)}}{\partial \theta_2^{(1)}} = \begin{pmatrix} x_1 \\ x_2 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ -2 \\ 1 \end{pmatrix}$$

In summary we obtain:

$$\frac{\partial L}{\partial \theta_2^{(1)}} = -0.762 \cdot 0.2 \cdot 0.221 \cdot \begin{pmatrix} 3 \\ -2 \\ 1 \end{pmatrix} = \begin{pmatrix} -0.101 \\ 0.067 \\ -0.034 \end{pmatrix}$$



Back propagation: example

Last step is the consideration of $\theta^{(2)}$ for which use the derivatives on slide 53:

$$\frac{\partial L}{\partial \theta^{(2)}} = \begin{pmatrix} \frac{\partial L}{\partial w_{11}^{(2)}} \\ \frac{\partial L}{\partial w_{12}^{(2)}} \\ \frac{\partial L}{\partial b_1^{(2)}} \end{pmatrix} = -2 \cdot (y - f(\mathbf{x}, \theta)) \cdot \begin{pmatrix} a_1^{(1)} \\ a_1^{(2)} \\ 1 \end{pmatrix} = -0.762 \cdot \begin{pmatrix} 0.475 \\ 0.67 \\ 1 \end{pmatrix} = \begin{pmatrix} -0.362 \\ -0.511 \\ -0.762 \end{pmatrix}$$



Back propagation: example

Combining the results of the previous three slides we obtain

$$\begin{aligned} -\nabla L(\boldsymbol{\theta}; \mathbf{x}_1, y_1) &= -\frac{\partial L}{\partial \boldsymbol{\theta}} \Big|_{\mathbf{x}_1, y_1} \\ &= -\left(\frac{\partial L}{\partial w_{11}^{(1)}}, \frac{\partial L}{\partial w_{12}^{(1)}}, \frac{\partial L}{\partial b_1^{(1)}}, \frac{\partial L}{\partial w_{21}^{(1)}}, \frac{\partial L}{\partial w_{22}^{(1)}}, \frac{\partial L}{\partial b_2^{(1)}}, \frac{\partial L}{\partial w_{11}^{(2)}}, \frac{\partial L}{\partial w_{12}^{(2)}}, \frac{\partial L}{\partial b_1^{(2)}} \right)^{\top} \\ &= (0.342, -0.228, 0.114, 0.101, -0.067, 0.034, 0.362, 0.511, 0.762)^{\top} \end{aligned}$$

Tataa !!!



Back propagation: example

This is done for all data i in our training data, e.g. $\mathbf{x}_2 = (-1, 1)^\top$ and $y_2 = 0.7$:

$$\begin{aligned}
 -\nabla L(\boldsymbol{\theta}; \mathbf{x}_2, y_2) &= -\frac{\partial L}{\partial \boldsymbol{\theta}} \Big|_{\mathbf{x}_1, y_1} \\
 &= -\left(\frac{\partial L}{\partial w_{11}^{(1)}}, \frac{\partial L}{\partial w_{12}^{(1)}}, \frac{\partial L}{\partial b_1^{(1)}}, \frac{\partial L}{\partial w_{21}^{(1)}}, \frac{\partial L}{\partial w_{22}^{(1)}}, \frac{\partial L}{\partial b_2^{(1)}}, \frac{\partial L}{\partial w_{11}^{(2)}}, \frac{\partial L}{\partial w_{12}^{(2)}}, \frac{\partial L}{\partial b_1^{(2)}} \right)^\top \\
 &= (0.04, -0.04, -0.04, 0.014, -0.014, -0.014, -0.155, -0.122, -0.27)^\top.
 \end{aligned}$$

Summing up gives

$$-\nabla L(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}) = -\sum_{i \in Tr} \nabla L(\boldsymbol{\theta}; \mathbf{x}_i, y_i),$$

the direction with the steepest descent w.r.t. to the current position on the high-dimensional landscape in the parameter space.



Summary: Back propagation

- ▶ Our neural net has a starting parameters $\theta = (w_{11}^{(1)}, w_{12}^{(1)}, \dots, b_1^{(2)})$.
- ▶ The cost for this starting point called the loss function $L(\theta)$.
- ▶ We iteratively find better position vectors θ using a steepest descent optimisation routine.
- ▶ Each iteration tells us to make one step downhill in the best direction.
- ▶ The best direction is $-\nabla L(\theta) = - \begin{pmatrix} \frac{\partial L}{\partial w_{11}^{(1)}} \\ \vdots \\ \frac{\partial L}{\partial b_1^{(2)}} \end{pmatrix}$.
- ▶ The magnitude of each component in this $\nabla L(\theta)$ tells which parameters are the most important in terms of reducing L .
- ▶ Each component of $\nabla L(\theta)$ is computed by repeatedly applying the chain rule.